케이녹 C언어 1회차 과제

17기 최동현 (202511688)

1-1. 변수와 자료형

• 변수 선언과 초기화

C언어 변수의 선언 :

```
int(자료형) a(변수명);
```

C언어 변수의 초기화 :

```
int b; // 변수 선언
b = 0(초기화할 데이터 값);
```

```c
#include <stdio.h>

int main(void) {
    int a;
    int b = 0;

    printf("a = %d\n", a);
    printf("b = %d\n", b);

    return 0;
}
```

```
~/Desktop/CHOI/KKnock  vim ./Day1/main.c

a = 1584661224
b = 0

계속하려면 엔터 혹은 명령을 입력하십시오
```

- ⋆**변수 초기화 생략 후 출력 시 쓰레기 값(Garbage Value)이 들어가는 이유** : 변수가 메모리 영역(Stack)을 할당 받을 때, 해당 그 공간이 이전에 프로그램이나 함수가 남겨놓은 데이터가 그대로 남아 있기 때문입니다.

1-2. 변수와 자료형

• 기본 자료형

C언어 기본 자료형 : int, char, float, double

```
// 기본 자료형
int(4바이트 정수형) a; // 1, 2, 3, ...
char(1바이트 문자형) b; // a, b, c, A, ...
float(4바이트 실수형) c; // 1.0, 1.1, 1.2, ...
double(8바이트 실수형) d; // float과 같음 (4Byte -> 8Byte)
```

float과 double의 가장 큰 차이점은 메모리에 할당 받는 크기가 총 4바이트에서 8바이트로 늘어난다는 차이가 있습니다.

- float 출력 서식 : %f
- double 출력 서식 : %lf

```
36%                                                    7%
1  #include <stdio.h>
1
2  int main(void) {
3      int a = 1;
4      char b = 'a';
5      float c = 3.14f;
6      double d = 3.14159;
7
8      printf("(int) a = %d\n", a);
9      printf("(char) b = %c\n", b);
10     printf("(float) c = %.2f\n", c);
11     printf("(double) d = %.5lf\n", d);
12
13     return 0;
14 }
```

```
36%                                                    7%
~/Desktop/CHOI/KKnock   vim ./Day1/main.c

(int) a = 1
(char) b = a
(float) c = 3.14
(double) d = 3.14159

계속하려면 엔터 혹은 명령을 입력하십시오
```

2-1. 연산자

- **산술 연산자** : 기본적인 수학 계산을 할 때 사용합니다.

```
// 산술 연산자
+ : 더하기
- : 빼기
* : 곱하기
/ : 나누기 값 (ex. 5 / 2 = 2)
% : 나눈 나머지 값 (ex. 5 % 2 = 1)
```

2-2. 연산자

- **대입 연산자**

  : 변수의 값을 지정할 때, 사용. 산술 연산자와 결합한 '복합 대입 연산자'도 자주 사용합니다.

```
// 대입 연산자
int a;
a = 1; // 오른쪽 값을 왼쪽 변수에 대입합니다.

// 복합 대입 연산자 : 연산 후 대입
a += 1; // 1 + 1 = 2 (a=1)
a -= 1; // 2 - 1 = 1
a *= 1; // 1 * 1 = 1
a /= 1; // 1 / 1 = 1
a %= 1; // 1 % 1 = 0
```

2-3. 연산자

- **증감 연산자**

  : 값을 1 증가시키거나 1 감소시킵니다. 연산자 위치에 따라 의미가 달라집니다.

```
// 증감 연산자
int a = 0;

++a; // a에 1을 먼저 더하고 해당 줄의 코드를 실행합니다.
a++; // 해당 줄의 코드를 실행하고 나중에 a에 1을 더합니다.
--a; // 위처럼 먼저 1을 빼고 실행
a--; // 코드 실행하고 나중에 a를 1을 감소
```

2-4. 연산자

- **관계 연산자**

  : 두 값을 비교하여 참(True : 1)인지 거짓(False : 0)인지를 알려줍니다.

```c
#include <stdio.h>

int main(void) {
    int a = 1;
    int b = 2;

    printf("(a==b) : %d\n", a==b);
    printf("(a!=b) : %d\n", a!=b);
    printf("(a>=b) : %d\n", a>=b);
    printf("(a<=b) : %d\n", a<=b);

    return 0;
}
```

```
~/Desktop/CHOI/KKnock  vim ./Day1/main.c

(a==b) : 0
(a!=b) : 1
(a>=b) : 0
(a<=b) : 1

계속하려면 엔터 혹은 명령을 입력하십시오
```

2-5. 연산자

- **논리 연산자**

  : 논리(Logical) 연산자로써 2진(0 or 1)체계를 사용합니다.

```c
#include <stdio.h>

int main(void) {
    int a = 0;
    int b = 1;

    printf("(a && b) : %d\n", a && b); // A와 B의 AND 연산 (a와 b, 둘다 참이면 1. 그렇지 않으면 0)
    printf("(a || b) : %d\n", a || b); // B와 B의 OR 연산 (a나 b, 둘중 하나만 참이여도 1. 둘다 거짓이면 0)

    return 0;
}
```

```
~/Desktop/CHOI/KKnock  vim ./Day1/main.c

(a && b) : 0
(a || b) : 1

계속하려면 엔터 혹은 명령을 입력하십시오
```

2-6. 연산자

- **비트 연산자**

  : 데이터 비트(0, 1) 단위로 직접 조작할 수 있도록 해줍니다. (중요! : 2진수 계산으로 함)

```c
#include <stdio.h>

int main() {
    int a = 1;      // 2진수로 0001
    int b = 2;      // 2진수로 0010

    printf("(& : AND) : %d\n", a & b);   // 둘 다 1이면 1 (0001 AND 0010 = 0000, 10진수로 0)
    printf("(| : OR) : %d\n", a | b);   // 하나라도 1이면 1 (0001 OR 0010 = 0011, 10진수로 3)
    printf("(^ : XOR) : %d\n", a ^ b);   // 서로 다르면 1 (0001 XOR 0010 = 0011, 10진수로 3)

    printf("(<<) %d\n", a << 1); // 왼쪽으로 1칸 밀기 (0001을 왼쪽으로 밀면 0010, 10진수로 2)
    printf("(>>) %d\n", a >> 1); // 오른쪽으로 1칸 밀기 (0001을 오른쪽으로 밀면 0000, 10진수로 0)

    return 0;
}
```

```
~/Desktop/CHOI/KKnock  vim ./Day1/main.c

(& : AND) : 0
(| : OR) : 3
(^ : XOR) : 3
(<<) 2
(>>) 0

계속하려면 엔터 혹은 명령을 입력하십시오
```

2-7. 연산자

- **삼항 연산자**

  : if문(조건)을 한 줄로 줄여 쓴 연산자입니다.

```c
#include <stdio.h>

int main() {
    int a = 1;
    int b = 2;
    int max = (a > b) ? a : b; // a가 b보다(a>b) 크면(?) a를 넣어라 아니라면(:) b를 넣어라

    printf("둘중 더 큰값 : %d\n", max);

    return 0;
}
```

```
~/Desktop/CHOI/KKnock  vim ./Day1/main.c

둘중 더 큰값 : 2

계속하려면 엔터 혹은 명령을 입력하십시오
```

3. 형 변환 (암시적 / 명시적)

- **암시적 변환**
  : 컴파일러가 알아서 형 변환

- **명시적 변환**
  : 개발자가 알아서 바꾸는 것

```
// 암시적 형 변환
float a = 10; // 컴파일러가 알아서 10을 10.0으로 초기화

// 명시적 형 변환
int b = (int)3.14; // 3.14를 정수값으로 형 변환하여 3으로 초기화
float c = (float)5/2; // 정수 나눗셈을 실수 나눗셈으로 변환하여 2.5로 초기화
```

4-1. 표준 입출력 (stdio.h)

- **표준 입출력 (printf / scanf)**

  : 콘솔 화면에 글자를 보여주거나(**printf**), 사용자의 입력장치(키보드 등)로 입력을받는(**scanf**) 함수입니다.

  꼭 ***#include <stdio.h>***로 표준 입출력 헤더(stdio.h)를 포함해야 합니다.

```c
#include <stdio.h>

int main() {
    int a = 0;

    printf("(입력) a = ");
    scanf("%d", &a);

    printf("(출력) a = %d\n", a);

    return 0;
}
```

```
~/Desktop/CHOI/KKnock  vim ./Day1/main.c

(입력) a = 3
(출력) a = 3

계속하려면 엔터 혹은 명령을 입력하십시오
```

4-2. 표준 입출력 (stdio.h)

- **서식 문자 (Format string)**

  : 어떤 형태의 데이터를 출력/입력할지 지정하는 약속된 기호들입니다.

```
// 서식 문자

%d // int : 정수형
%f // float : 실수형
%lf // dobule : 실수형 8바이트 (scanf에서 double 사용 시 필수!)
%c // char : 문자 1개
%s // 단어나 문장(string)
%p // 포인터(*) : 메모리 주소값
```

4-3. 표준 입출력 (stdio.h)

- **이스케이프 시퀀스 (Escape sequence)**

  : 탈출이라는 말처럼 기능에 사용되는 특수문자를 출력하거나, 특수한 기능을 할 수 있도록 하는 문자들로 Backslash(₩)를 사용합니다.

```
// 이스케이프 시퀀스

\n // 줄 바꿈
\t // 탭 만큼 띄우기
\\ // 백슬레시 출력
\" // 큰 따옴표 출력

 ...
```

감사합니다

17기 최동현